后训练学习计划 • 第 8 周（Phase 2 • 偏好优化：手推 Bradley-Terry 与 DPO，跑通第一条 DPO 链路，10/26-11/1）

本周定位：**从「有标答的监督学习」跨到「没有标答、只有比较」的偏好优化**。手推 Bradley-Terry 与 DPO 到能在白纸上讲满 3 分钟；用强模型把第 6 周 SFT 模型的记忆口诀输出标成 1000-2000 对偏好数据（标注提示词版本化 + 100 条人工复核一致率）；用 `DPOTrainer`（β=0.1、lr=5e-6）真跑一遍；评测同时给 held-out 偏好对准确率与 20 题人工盲评，并专门盯 reward hacking。

用法说明：每天先看「今日总览」，再从任务 1 逐条做；每个任务都有 **完成判据**，判据不满足不要进入下一个任务。**加粗 = 硬性要求**，灰色小字块 = 注释（可跳过，不影响主线）。

一、本周总览

天	主题	核心硬产出	预计
Day 1 周一	环境与偏好数据格式 + Bradley-Terry 手推	BT 损失 numpy 版与 torch 版对拍 OK	6h
Day 2 周二	DPO 闭式解推导 + IPO/KTO/SimPO/ORPO	推导手稿六步 + 四方法差异表	6h
Day 3 周三	采候选 + 强模型打分 + 配对	1000-2000 对偏好数据落盘（含标注提示词）	6h
Day 4 周四	100 条人工复核 + held-out 切分 + mini-DPO	人工一致率数字 + mini-DPO 单测全绿	6.5h
Day 5 周五	DPOTrainer 正式训练（β=0.1、lr 5e-6）	DPO adapter + 训练日志（margin 上行）	6h
Day 6 周六	评测：held-out 准确率 + 20 题盲评 + hacking 观察	两份评测表 + 长度分布对照	7h
Day 7 周日	补漏 + 闭卷复测 + 总验收 + 下周预习	验收 ≥6/7 项通过	4.5h

注：**本周卡时上限 12h（4090 24G），GPU 预估 ≈¥25**（¥1.5-2.5/时，按 12h×2.5=¥30 封顶；超了第一件事去看 AutoDL 计费明细，通常是忘关机）。分配：Day 3 采样 2h、Day 5 训练 5h、Day 6 生成 3h、留 2h 给 Day 5 重跑。**API 另计 ≈¥20**（Day 3 打分 + Day 6 判卷），本周合计上限 ¥50。

注：**本周最大风险是 Day 3 的偏好数据质量，不是训练**。DPO 是离线方法，对噪声标注的容忍度远低于对超参的容忍度——一致率低的偏好数据训出来的模型，学到的是标注器的抖动。所以 Day 4 的 100 条人工复核是硬关卡，不许跳。砍单顺序（做不完按这个砍）：Day 4 任务 3 mini-DPO → Day 6 任务 3 的套话统计（保留长度分布）→ Day 2 任务 5 论文笔记 → **Day 3 标注、Day 4 复核、Day 5 训练、Day 6 两份评测表不许砍**。

注：本周不做 GRPO、不碰 PPO、不装 verl，全部留给下周。今天讲不清 DPO「为什么不需要 RM」，下周 GRPO 的组内基线推导就会滑成背书。

二、开始前：环境 / 数据 / 账号（约 50 分钟，前一晚做完）

1. **本地 venv（不要装在 conda base 里）**：

```
cd E:/study/posttrain && python -m venv .venv-dpo
source .venv-dpo/Scripts/activate                # Git Bash；PowerShell 用 .venv-dpo\Scripts\Activate.ps1
pip config set global.index-url https://pypi.tuna.tsinghua.edu.cn/simple
pip install -U "trl>=0.11" "transformers>=4.45" "peft>=0.13" accelerate datasets numpy matplotlib
python -c "import torch, trl, transformers; print(torch.__version__, trl.__version__,
transformers.__version__)"
```

> 注：**Windows + Anaconda 必踩**：`import torch` 报 `OMP: Error #15: Initializing libiomp5md.dll, but found libiomp5md.dll already initialized`，根因是 conda 的 MKL 与 torch 自带的 OpenMP 各加载一份。正解就是上面这个独立 venv；急着往下做先临时绕过：cmd `set KMP_DUPLICATE_LIB_OK=TRUE`，Git Bash `export KMP_DUPLICATE_LIB_OK=TRUE`。**别用 `pip install --force-reinstall` 折腾**，那是同一个坑的第三种踩法。`bitsandbytes` 在本机常装不上，**本周不需要它**（QLoRA 上周已过），上卡再装。

2. 建目录、核对第 6/7 周产物在位，再把第 6 周模型 merge 成完整权重（DPO 从 SFT 权重起步，ref 也用它）：

```
mkdir -p E:/study/posttrain/week08/{day1,day2,day3,day4,day5,day6,day7}
ls E:/study/posttrain/llm-posttraining-lab/m2-sft/data          # 上周 instruct 数据必须在位
python -c "from peft import PeftModel; from transformers import AutoModelForCausalLM; \
m=AutoModelForCausalLM.from_pretrained('E:/study/posttrain/qwen15b', torch_dtype='auto'); \
m=PeftModel.from_pretrained(m, 'E:/study/posttrain/llm-posttraining-lab/m2-sft/adapter'); \
m.merge_and_unload().save_pretrained('E:/study/posttrain/week08/sft-15b-merged')"
```

> 注：merge 后目录约 3GB（1.5B × bf16 两字节/参）。上周若你只跑了 0.5B，本周主线就落在 0.5B、1.5B 变 Day 5 的冲刺项——**规模降一级不算破功，跳过头顶的推导才算**。

3. 账号与余额：AutoDL ≥¥50（本周 ¥25 + 下周先充）；打分 API ≥¥25，**批量打分前先看一次定价页，别用记忆里的价格算账**。HF 打不开走 `export HF_ENDPOINT=https://hf-mirror.com`，或魔搭直下 `modelscope download --model Qwen/Qwen2.5-1.5B --local_dir ./qwen15b`。

4. 上周 gaps 清账：本周 Day 1 只清这三类洞（其余不动）：① SFT 的 label=-100 与 chat template（周 6）② 去污染 n-gram 脚本（周 7）③ LoRA 的 rank/alpha 为什么这么选（周 6）。它们分别是 Day 1 任务 4、Day 4 任务 2、Day 5 任务 1 的前置。

Day 1 • 周一：Bradley-Terry 与奖励模型的地基

今日总览（6h）：今天不训练、不开卡。三件事：装好 DPO 依赖并把版本号记账（版本漂移是 DPO 报错的头号来源）；**手推 Bradley-Terry 损失并独立实现一遍，与 torch 对拍**；把偏好数据的字段格式核对清楚。晚上你要能不看资料答出「为什么后训练用成对比较而不是绝对打分」。

任务 1：环境与产物体检（约 40 分钟）

1. 激活 venv，把五个版本号写进 `week08/day1/versions.txt`（`torch / transformers / trl / peft / datasets` + `torch.cuda.is_available()`），一行一条。
2. 确认这三句都能 import 成功：`from trl import DPOTrainer, DPOConfig`、`from peft import LoraConfig, get_peft_model`、`from datasets import load_dataset`。

完成判据：`day1/versions.txt` 存在且五个版本号齐全；三条 import 不报错。**后面所有报错都靠这份版本号对照**——网上抄来的代码用的是另一版 API，这是常态不是意外。

任务 2：手推 Bradley-Terry 损失（约 110 分钟）

1. 在纸上推这四步，每步写清「用了什么假设」，整理成 `day1/bt.md`：
   - 假设一：回答质量可用一个实数 r(x,y) 表示。
   - 假设二：`p(y_w > y_l | x) = σ(r(x,y_w) - r(x,y_l))`。**只有差有意义 → 奖励允许平移 r+c、缩放 r/c，所以奖励是相对的、不能跨 prompt 比大小。**
   - 假设三：N 个偏好对相互独立 → 联合似然是各对概率之积 → 负对数似然就是求和。
   - 得 **L_BT = −E[log σ(r(x,y_chosen) − r(x,y_rejected))]**。写出它对 r_w、r_l 的梯度（形如 `-σ(-(r_w-r_l))·[+1, -1]`），并说明它本质是 sigmoid 上的二元交叉熵、标签恒为「chosen 赢」。
2. numpy 实现并与 `F.binary_cross_entropy_with_logits` 对拍，**统一 float64**：

```
# week08/day1/bt_loss.py — Bradley-Terry 损失: numpy 手写 vs torch 内建
import numpy as np, torch
rng = np.random.default_rng(0); torch.manual_seed(0)          # seed 写死: 对拍要逐位可复现
r_w, r_l = rng.normal(0.6, 1.2, 512), rng.normal(0.0, 1.2, 512)
diff = (r_w - r_l).astype(np.float64)                          # float64: 容差 1e-9 在 float32 上会被噪声吞掉
def sigm(z):                                                   # 只让 exp 的指数 ≤0, 防上溢; np.where 两支共用同一个 e, 不会重复算
    e = np.exp(-np.abs(z)); return np.where(z >= 0, 1.0/(1.0+e), e/(1.0+e))
bt = -np.log(np.clip(sigm(diff), 1e-12, 1.0)).mean()           # clip 必需: σ 饱和时 log(0)=nan
ref = torch.nn.functional.binary_cross_entropy_with_logits(
    torch.tensor(diff, dtype=torch.float64), torch.ones(len(diff), dtype=torch.float64)).item()
print("bt_numpy =", round(float(bt), 10), " bt_torch =", round(ref, 10))
assert abs(bt - ref) < 1e-9, "BT 对拍失败"; print("BT CROSSCHECK OK")
```

3. 把 `r_w` 换成与 `r_l` 同分布（奖励无信号）跑一次、再把 `diff` 整体减 3（人为拉开）跑一次，**两个 loss 数值都写进 NOTES.md，不许只写「变小了」**。

完成判据：打印 `BT CROSSCHECK OK` 且差 <1e-9；NOTES.md 有那两次改数据的 loss 数值 + 一句「为什么 r 的常数平移不影响 BT」。

> 注：无信号时 BT loss 在 ln2≈0.693 量级（σ(0)=0.5），拉开差距后往 0 走。记住这两个量级，面试问「BT loss 一开始应该是多少」就不用猜。

任务 3：RM = 基座 + 回归头，以及三个必被追问的点（约 80 分钟）

1. 在 NOTES.md 画 RM 结构：基座最后一层 hidden（B,T,C）→ 取每条**真正的末尾有效 token**（不是最后一个 token，右填充要靠 attention_mask 反推）→ `Linear(C, 1)` → 标量。跑通取位代码：

```
import torch
hidden = torch.randn(2, 7, 4); attn = torch.tensor([[1,1,1,1,1,0,0],[1,1,1,1,1,1,1]])
idx = attn.sum(1) - 1                                          # 每条各自的最后一个有效 token 下标
assert hidden[torch.arange(2), idx].shape == (2, 4); print("POOLING OK")
```

2. 三问三答各 ≥3 行写进 NOTES.md：① RM 打分打在「回答」还是「整段对话」？② RM 会被训坏吗、坏在哪（**reward hacking 的定义 / 发现手段 / 缓解手段各一行**）？③ 为什么后训练团队要定期用新策略的输出重训 RM?

完成判据：打印 `POOLING OK`；NOTES.md 三问答案齐全，第 ② 问含「监控输出长度分布突变」这一条具体手段。

> 注：第 ③ 问就是 DPO 存在的一半理由——RM 是被策略「攻击」的对象，DPO 干脆不训练这个会被攻击的裁判。这条因果链写清楚，Day 2 的推导就不是新知识，只是把它写成数学。

任务 4：偏好数据格式与 TRL 字段核对（约 60 分钟）

1. 定死本周偏好数据一行长什么样（jsonl，一行一对）：

```
{"prompt": "用一句话给 ambivalence 编一个中文记忆口诀，要能一眼记住", "chosen": "音似「俺不能走」—爱也不是恨也不是，卡在中间那个「俺」就是犹豫。", "rejected": "ambivalence 表示矛盾情感，前缀 ambi 意为双方，所以是两种情感并存。", "score_chosen": 4.4, "score_rejected": 2.8, "src": "day3-batch-0007"}
```

2. 打印本机 `DPOConfig` 的签名存 `day1/dpoconfig.txt`，把 `beta`、`learning_rate`、`max_length`、`truncation_mode`、`loss_type` 五个字段的**默认值**抄进 NOTES.md：

```
python -c "import inspect, trl; print(inspect.signature(trl.DPOConfig.__init__))" >
E:/study/posttrain/week08/day1/dpoconfig.txt
```

3. 写个小脚本读 3 行 jsonl，断言三字段齐全、chosen ≠ rejected、两者非空，通过则打印 `FORMAT OK`。

完成判据：`dpoconfig.txt` 存在；校验脚本打印 `FORMAT OK`；NOTES.md 抄全五个字段的默认值（**不许写「大约」**）。

> 注：TRL 的偏好数据集支持两种格式——纯字符串列，和 conversational（每列是 `{"role","content"}` 列表）。**本周选纯字符串列，模板自己用 `apply_chat_template` 拼**，因为 Day 6 你要精确控制 assistant span 做 logprob 掩码，conversational 把模板藏在 trainer 内部，出问题时你连差异在哪都找不到。代价是多写十行，值得。

任务 5：标注 prompt 池准备（约 70 分钟）

从第 7 周数据集分层抽 600 条 instruction 当 prompt 池（记忆口诀任务），**按主题分桶等比例抽**，别全抽同一个词类：

```
# week08/day1/build_pool.py
import json, random
random.seed(2026)                                          # 写死：Day 3 重跑必须是同一批 prompt
rows = [json.loads(l) for l in open("E:/study/posttrain/llm-posttraining-lab/m2-sft/data/instruct.jsonl",
encoding="utf-8")]
by = {}
for r in rows: by.setdefault(r.get("topic", "unknown"), []).append(r)
pool = []
for t, items in sorted(by.items()):                        # 按主题分桶等比例抽，别全抽同一个词类
    pool += random.sample(items, min(max(1, round(600*len(items)/len(rows))), len(items)))
random.shuffle(pool); pool = pool[:600]
with open("E:/study/posttrain/week08/day1/pool600.jsonl", "w", encoding="utf-8") as f:
    for p in pool: f.write(json.dumps(p, ensure_ascii=False)+"\n")
print("pool =", len(pool), "| topics =", len(by))
```

另把第 6 周那 20 道固定评测题存成 `pool20_eval.jsonl`，并写一句断言：两者交集打印为 0。

完成判据：`pool600.jsonl` =600 行（不足就补抽并在 NOTES 记原因）；主题数 ≥5；`pool20_eval.jsonl` =20 行，交集脚本打印 0。

> 注：「交集为空」不是洁癖。20 题是 Day 6 盲评的题面，若它出现在 Day 3 的偏好训练数据里，你的盲评就是在给模型看过的题打分，结论全盘作废。**去污染从今天起是每次造数据的第一动作，不是最后一道检查。**

今日验收：versions.txt + `BT CROSSCHECK OK` + `POOLING OK` + `FORMAT OK` + pool600 与交集 0 + NOTES 新增 ≥6 条。

Day 2 • 周二：DPO 闭式解推导 + 四个「后 DPO」方法

今日总览（6h）：本周理论重心。目标很具体：**明天早上你能在一张白纸上从 RLHF 目标函数写到 DPO loss，六步，每步说清用了什么假设，讲满 3 分钟不卡壳。**下午把 IPO/KTO/SimPO/ORPO 各自解决什么问题理清，闭卷默写过关。

任务 1：六步推导（约 120 分钟）

按顺序推，每步一段，整理成 `week08/day2/dpo-derivation.md`（纸上手稿 Day 7 拍照入库）：

1. RLHF 目标：`max_π E_{x, y~π}[ r(x,y) ] − β · KL( π(y|x) ‖ π_ref(y|x) )`。β 一句话：**一条绳子，防止策略为了骗 reward 跑太远而语言崩坏。**
2. 在 Σπ=1 约束下对 π 求变分极值（拉格朗日），得闭式最优策略 `π*(y|x) = (1/Z(x)) · π_ref(y|x) · exp( r(x,y)/β )`，`Z(x) = Σ_y π_ref(y|x)exp(r(x,y)/β)`。**注意 Z(x) 只依赖 x、不依赖 y**——这是下一步能消掉它的唯一原因。
3. 取对数反解奖励：`r(x,y) = β · log( π*(y|x)/π_ref(y|x) ) + β · log Z(x)`。**这一步是 DPO 的全部魔法：奖励被写成「策略与参考策略的对数概率比」。**
4. 代进 Bradley-Terry `p(y_w ≻ y_l) = σ(r(y_w) − r(y_l))`，`β log Z(x)` 在差里**直接相消**（这就是那个算不动的配分函数不用管的原因）。

5. 得 **DPO loss**：`L_DPO = -E[ log σ( β·( log π_θ(y_w|x) - log π_ref(y_w|x) - log π_θ(y_l|x) + log π_ref(y_l|x) ) ) ]`。顺带定义 **implicit reward** `r̂(x,y) = β log(π_θ(y|x)/π_ref(y|x))`，于是 loss = `-log σ( r̂(y_w) - r̂(y_l) )`，与 BT 同形。
6. 三个落点各 2-3 行：① **为什么不需要 RM**（裁判被代数消掉了）② **β 太大/太小**（太大 → 贴着 ref 几乎不更新，学不到偏好；太小 → 过拟合偏好、语言退化）③ **整句概率是逐 token 求和**，`log π(y|x) = Σ_t log π(y_t|y_<t,x)`，所以长度天然影响量级。

完成判据：`dpo-derivation.md` 有六个步骤段落，第 2、3、4 步各标出「用了什么假设」；NOTES.md 抄上第 5 步的 loss 全式。

> 注：第 6 步第 ③ 点下周直接变成分数：GRPO 的 KL 按 token 平均，而 DPO 对整句求和，这是 DPO 偏爱长回答的一个来源，也是 SimPO 改用长度归一化的动机。**今天多写这一行，Day 2 任务 3 就不用重新想。**

任务 2：implicit reward margin 的可观测量（约 60 分钟）

把推导里定义的三个量做成**训练时要打印的指标**，先在 NOTES.md 默写这张表（写完再对照）：

指标	公式	健康表现	出问题时说明什么	本周哪个任务能观察到
chosen logprob 差	`log π_θ(y_w|x) - log π_ref(y_w|x)`	缓慢上行	大幅上行而 margin 不涨 → 只把「好回答」学得更顺，没学会比较	Day 5 任务 3
rejected logprob 差	`log π_θ(y_l|x) - log π_ref(y_l|x)`	缓慢下行	也在上行 → 两边都在加分，只是 chosen 加得快，这是 DPO 常见形态，**别当 bug**	Day 5 任务 3
margin	上两者之差	上行	不涨 → 数据没信号或 lr 太小	Day 5 任务 3

完成判据：表格三行指标齐全、每格有内容，最后一列指向具体任务号（**不许写「看日志」，要写「Day 5 任务 3 从 wandb 读含 margin 的指标」**）。

> 注：TRL 的 DPO 日志常见字段是 `rewards/chosen`、`rewards/rejected`、`rewards/margins`、`rewards/accuracies`、`logps/chosen`、`logps/rejected`，**各版本前缀略有差异。别硬编码字段名**：跑起来后把 wandb 指标列表全打一遍对着认。这 5 分钟省掉你后来 1 小时「为什么我的曲线是空的」。

任务 3：IPO / KTO / SimPO / ORPO 各解决什么（约 90 分钟）

写四行 × 四列表进 `week08/day2/alt-methods.md`，列是：**针对 DPO 的哪个具体问题 / 核心改动一句话 / 代价 / 面试里怎么讲**：

1. **DPO 过拟合**：偏好数据一旦接近确定性（chosen 与 rejected 差得很开），DPO 会把 margin 推向无穷、脱离 ref → **IPO** 把 margin 拉向一个有限目标值，防止奖励差无限扩张。
2. **必须要有成对数据**：真实标注常常只有「这条好 / 这条差」的单边标签 → **KTO** 用前景理论（人对得失的敏感度不对称）直接吃二元标签，不需要同 prompt 配对。
3. **长度偏置与 serving 不一致**：DPO 概率是整句求和，长回答天然分高；且训练用 ref、推理没有 ref → **SimPO** 用长度归一化的平均 logprob 当奖励、彻底去掉 ref model。

4. **SFT 与偏好优化两阶段的成本 → ORPO** 在同一次 SFT 里加一个 odds ratio 惩罚项，一阶段完成对齐。

完成判据：四行四列无空格；口头连讲四个各 ≤40 秒，**不许出现「也是一种改进方法」这类无信息句**（自己录音听一遍，120 秒内讲完四个）。

> 注：简报只要求讲清 3 个，第 4 个是为面试那句「还有别的吗」准备的。**加分动作**：把这四个与 DPO 排成「需不需要 RM / 需不需要在线采样 / 显存几份权重 / 稳定性」四列对照，那是周 14 八股的现成成品，今天多花 10 分钟。

任务 4：闭卷默写（约 50 分钟）

合掉所有窗口，在 `day2/recall.md` 闭卷写三题：① SFT loss 与 label mask 规则（上周该闭合的，今天验收）② BT loss +「为什么用成对比较不用绝对打分」③ DPO loss 全式（含 β）+ implicit reward 定义 +「为什么不需要 RM」三句话。写完逐字批改，**错一个符号也标出并数清个数**。

完成判据：三题全写；DPO 全式无漏项、β 位置正确；mask 规则写出「instruction 与模板 token 都置 -100，只 assistant 内容参与」。允许 ≤2 个符号级小错，**结构性漏项（少一整项 / 丢了 β / 没说 Z 相消）判为不过，当晚重默一遍**。

任务 5：论文与资源（约 40 分钟，可砍到 1 条）

读 DPO 论文的 derivation 一节 + DeepSeekMath 里 GRPO 那一节开头，各写 100 字进 `notes/papers.md`。本周精读配额两篇：DPO 与 InstructGPT（上周留的尾巴，只读 RLHF 三步那节）。

完成判据：`notes/papers.md` 新增 2 条各 ≥100 字（砍掉本任务的一半则 ≥1 条）。

今日验收：六步推导齐全 + 四方法表无空格 + 闭卷默写判为「过」+ NOTES 新增 ≥5 条。

Day 3 • 周三：造偏好数据（采候选 → 强模型打分 → 配对）

今日总览（6h）：本周最花钱、最容易返工的一天。上午把标注提示词与 rubric 定死并**先算清账**；中午开卡 2 小时用第 6 周 SFT 模型对 600 个 prompt 各采 4 个候选；下午用强模型给 2400 条打分、同 prompt 内取 max-min 配对，晚上落盘 ≥1000 对。**标注提示词从这一刻起是项目资产，必须版本化保存。**

任务 1：标注提示词 + rubric + 算账（约 80 分钟）

1. 写 `day3/score_prompt.txt`（**v1，之后每次改动加版本号与日期，不许覆盖**）：**低温、纯 JSON、四条 rubric 各 1-5 分**：

```
你是中文英语学习产品的记忆口诀质量评审。给定一个场景与一条候选口诀，按四条维度各打 1-5 整数分。
【场景】{prompt}    【候选口诀】{candidate}
- accuracy：与词义、拼写、读音是否真的对应，有没有编错
- mnemonic：能否一眼记住（音似/形似/画面钩子），而不是把释义重说一遍
- concise：≤25 字 5 分，26-40 字 3 分，>40 字 1 分
- no_filler：不含「我完全理解你的感受」「希望这能帮到你」这类填充套话
总分 = 0.35*accuracy + 0.35*mnemonic + 0.15*concise + 0.15*no_filler
只输出 JSON：{"accuracy":int,"mnemonic":int,"concise":int,"no_filler":int,"total":float,"reason":"≤25字理由"}
```

2. 定死采样配置 `day3/sample_config.json`：`n=4`、`temperature=0.9`、`top_p=0.95`、`max_tokens=200`、`seed=2026`。
3. **开跑前把账写在 NOTES.md**（不许跑完再算）：

```
打分调用数 = 600 prompt × 4 候选 = 2400 次
单次 token 量级：输入 ≈1.2k（含场景+口诀）+ 输出 ≈0.12k → 合计 ≈2.9M 输入 / 0.3M 输出
用定价页实际单价乘出金额并写下：¥____
判定：≤¥20 直接跑；¥20-35 候选 4→3（仍要 ≥1000 对）；>¥35 换更便宜的打分器，但 Day 4 人工复核升到 150 条。
```

完成判据：`score_prompt.txt` 与 `sample_config.json` 存在；NOTES.md 有**用真实单价算出的**金额数字（不是「应该不贵」）；四条权重相加 =1.0。

> 注：`temperature=0.9` 是给**被评模型**的（要多样性才拉得开差距），打分器必须 **temperature=0**（要可复现）。这两个值弄反是典型事故：打分器一热，同一条候选两次跑出不同分，**你后面所有「一致率」都成了噪声的度量**。

任务 2：开卡采 2400 条候选（约 110 分钟，用卡 2h）

1. AutoDL 开 4090（镜像 pytorch2.x + cuda12.1 + py3.10），JupyterLab 终端装 vLLM，把 `sft-15b-merged`（或 0.5B 版）与 `pool600.jsonl` 传上**持久盘** `/root/autodl-tmp`（`/root` 下可能随镜像重置丢）：

```
source /etc/network_turbo                    # 学术加速，对 github/hf/pypi 有效
pip install -U vllm -i https://pypi.tuna.tsinghua.edu.cn/simple
mkdir -p /root/autodl-tmp/week08 && cd /root/autodl-tmp/week08
```

```
# week08/day3/sample.py — 600 prompt × 4 候选
from vllm import LLM, SamplingParams
import json
llm = LLM(model="/root/autodl-tmp/sft-15b-merged", max_model_len=2048, dtype="bfloat16", seed=2026)
tok = llm.get_tokenizer()
pool = [json.loads(l) for l in open("pool600.jsonl", encoding="utf-8")]
msgs = [tok.apply_chat_template([{"role":"user","content":p["instruction"]}],
        tokenize=False, add_generation_prompt=True) for p in pool]
outs = llm.generate(msgs, SamplingParams(n=4, temperature=0.9, top_p=0.95, max_tokens=200, seed=2026))
with open("candidates.jsonl", "w", encoding="utf-8") as f:
    for o, p in zip(outs, pool):
        for i, c in enumerate(o.outputs):
            f.write(json.dumps({"prompt": p["instruction"], "idx": i, "text": c.text.strip()},
ensure_ascii=False)+"\n")
print("candidates =", sum(len(o.outputs) for o in outs))
```

完成判据：`candidates.jsonl` =2400 行（因长度截断 <2400 可接受，但 ≥2340 并记缺口原因）；每行 `text` 非空；日志无 OOM。

> 注：用 `apply_chat_template` 而不是手拼字符串——ChatML 那套特殊标记自己拼很容易错一位，模型输出立刻退化，而你会去查温度。**上周 gaps 里「chat template」那条就是为今天准备的。**
> 注：vLLM 与镜像里的 torch 打架（`ImportError` / `undefined symbol`）时，降级顺序：① `pip install vllm==<镜像 torch 对应版本>`；② 改用 HF 的 `model.generate(num_return_sequences=4, do_sample=True)`（慢一个数量级，600×4 约 60-80 分钟，就把任务 3 挪到明早）；③ 砍到 300 prompt。**别让它吃掉整天的预算，最多 30 分钟就换道。**

任务 3：强模型打分 2400 条（约 110 分钟）

带断点续跑的批量打分，**结果一条一落盘，中断不重头**：

```
# week08/day3/score.py
import json, os, time
from openai import OpenAI
cli = OpenAI(api_key=os.environ["LLM_API_KEY"], base_url=os.environ["LLM_BASE_URL"])
TPL = open("score_prompt.txt", encoding="utf-8").read()
done = {json.loads(l)["uid"] for l in open("scores.jsonl", encoding="utf-8")} if
os.path.exists("scores.jsonl") else set()
with open("scores.jsonl", "a", encoding="utf-8") as fo:      # 追加写：中断后重跑只补未完成的 uid
    for c in [json.loads(l) for l in open("candidates.jsonl", encoding="utf-8")]:
        uid = f'{c["prompt"][:40]}#{c["idx"]}'
        if uid in done: continue
        for attempt in range(3):                              # 3 次就放弃并记录，别死磕
            try:
                r = cli.chat.completions.create(model=os.environ["SCORE_MODEL"], temperature=0,
                    response_format={"type":"json_object"},
                    messages=[{"role":"user","content":TPL.format(prompt=c["prompt"],
candidate=c["text"])}])
                fo.write(json.dumps({"uid": uid, **c, **json.loads(r.choices[0].message.content)},
ensure_ascii=False)+"\n")
                break
            except Exception as e:
                print("retry", attempt, type(e).__name__, e); time.sleep(2 + 2*attempt)
print("scored =", sum(1 for _ in open("scores.jsonl", encoding="utf-8")))
```

```
export LLM_API_KEY=sk-xxxx            # 别写进代码，也别 commit
export SCORE_MODEL=<你选的打分模型名>
python score.py 2>&1 | tee score.log
```

完成判据：`scores.jsonl` ≥2200 行；`total` 全部可转 float；≥90% 行带非空 `reason`；把累计 token 数与任务 1 的预算对照一次（**超了就停下重算，别继续烧**）。

> 注：`response_format` 各家支持度不同，报错就去掉这个参数，改成 `re.search(r"\{.*\}", txt)` 抠 JSON，并把**解析失败率**打出来（>5% 就换打分模型——不是你的错，是模型不守格式）。429 是并发问题不是账单问题：并发压到 4-8、指数退避。**别写成 `while True` 无限重试**，一晚上跑挂的循环能把余额吃空。

任务 4：max-min 配对与落盘（约 40 分钟）

只在同一 prompt 内比较（跨 prompt 的分数不可比，这是 Day 1 假设二的直接推论）：

```
# week08/day3/pair.py — 每个 prompt 内取 (max,min) 与 (2nd max,min)
import json, collections
by = collections.defaultdict(list)
for r in [json.loads(l) for l in open("scores.jsonl", encoding="utf-8")]: by[r["prompt"]].append(r)
pairs, skipped = [], 0
for p, items in by.items():
    if len(items) < 3: skipped += 1; continue
    items.sort(key=lambda x: x["total"], reverse=True)
    top, second, worst = items[0], items[1], items[-1]
    if top["total"] - worst["total"] < 0.6: skipped += 1; continue    # 差距太小 = 无信号，丢掉
    pairs.append({"prompt": p, "chosen": top["text"], "rejected": worst["text"],
                  "score_chosen": top["total"], "score_rejected": worst["total"], "pair_type": "max_min"})
    if second["total"] - worst["total"] >= 0.6 and second["text"] != top["text"]:
        pairs.append({"prompt": p, "chosen": second["text"], "rejected": worst["text"],
                      "score_chosen": second["total"], "score_rejected": worst["total"], "pair_type":
"second_min"})
with open("pref_pairs_raw.jsonl", "w", encoding="utf-8") as f:
    for x in pairs: f.write(json.dumps(x, ensure_ascii=False)+"\n")
print("pairs =", len(pairs), "| skipped =", skipped, "| prompts =", len(by))
```

完成判据：`pref_pairs_raw.jsonl` 落在 **1000-2000 对**。不足 1000 就回任务 2 把 4 个候选补到 6 个，或把阈值 0.6 降到 0.45，**并在 NOTES 记录你改的是哪一个**；`pair_type` 两类都有；同一 prompt 出现在 pair 里不超过 2 次。

> 注：阈值 0.6 是刻意设的「无信号就丢」。宁可少 200 对，也不要 500 对「chosen 和 rejected 其实一样好」的数据——DPO 处理这种对的方式是把 margin 拼命往上推，**学出来的是抖动而不是偏好**。这条是 Day 6 hacking 观察的伏笔。

任务 5：关机与收工（约 20 分钟）

去 AutoDL 控制台确认状态显示「已关机」（**关机 ≠ 销毁，任何时候不点销毁**），NOTES.md 记今日卡时（应 ≈2h）、API 实际花费、pair 数、跳过的 prompt 数。

完成判据：控制台状态为已关机；NOTES.md 有当日卡时与花费两个数字。

今日验收：candidates 2400 + scores ≥2200 + pref_pairs_raw 在 1000-2000 区间 + 算账数字落纸 + 已关机。

Day 4 • 周四：人工复核一致率 + held-out 切分 + mini-DPO

今日总览（6.5h）：把昨天的标注变成可信资产。上午**人工复核 100 条并给出一致率数字**（这是本周最值钱的面试素材，也是唯一一步质检）；中午做 held-out 切分与去污染；下午**手写一份 mini-DPO 并用它对拍 DPO 的语义**——两偏好对 + 两个玩具网络，float64、梯度中心差分校验。今天结束你要能回答「你的偏好数据可信度是多少、怎么量的」。

任务 1：100 条人工复核（约 110 分钟）

```
# week08/day4/review_export.py — 导出人判表，并删掉机器分列避免锚定
import json, random
random.seed(2026)
pairs = [json.loads(l) for l in open("pref_pairs_raw.jsonl", encoding="utf-8")]
sample = random.sample(pairs, min(100, len(pairs)))
with open("review100.md", "w", encoding="utf-8") as f:
    f.write("| # | prompt | chosen | rejected | 人判(选对/差不多/选反) | 备注 |\n|---|---|---|---|---|---|\n")
    for i, p in enumerate(sample, 1):
        f.write(f"| {i} | {p['prompt'][:40]} | {p['chosen'][:60]} | {p['rejected'][:60]} |  |  |\n")
print("rows =", len(sample))
```

1. **逐条自己判**：表里**不给机器分**（上面已删列），只看两个答案选更好的；判完再与机器配对对照，填三类之一：**选对** / **差不多** / **选反**。
2. 算三个数写进 `m2-sft/dpo/annotation-report.md`：`一致率 = 选对/100`；`强一致率 = (选对 − 0.5×差不多)/100`（把「差不多」折半，标注行业常用的粗算口径）；`差不多占比`。
3. 另记两条诊断：差不多占比 >25% 说明阈值太松；「选反」的 3 个典型原因（哪条 rubric 最不可信）。

完成判据：`review100.md` 的 100 行人判列**全部填完（空白 = 没做）**；报告里三个数字齐全。门槛：**一致率 ≥70%** 才允许直接进 Day 5；**60-70%** → 只保留「人判=选对 且 机器分差 ≥1.0」的对进训练，并记录筛掉了多少对；**<60%** → 改 `score_prompt.txt` 的 rubric 重跑 200 条再复测（**这是一次真实返工，写进 gaps**）。

> 注：面试官在这里会追两层——「一致率怎么算的」和「一致率不高你会做什么」。**只记一个百分比不够，第二个数字（筛选后剩下的对数）才证明你真的懂数据。**
>
> 注：LLM-as-judge 有两个成体系的偏置：**位置偏置**（倾向选第一个给出的答案）与**长度偏置**（把「更长」当「更好」）。本方案靠三条压它：① 只在同 prompt 内比较 ② 打分器 temperature=0 ③ Day 6 盲评显式看长度。**你自己复核时的偏置也在，别假设「人工」就干净**——所以人判时先遮住机器分。

任务 2：held-out 切分 + 去污染（约 90 分钟）

按 prompt 切 90/10（**同一 prompt 的所有 pair 必须同侧**，否则 held-out 形同虚设），并与第 6/7 周用过的题面做 13-gram 交集剔除：

```
# week08/day4/split_clean.py
import json, random
random.seed(2026)
pairs = [json.loads(l) for l in open("pref_pairs_raw.jsonl", encoding="utf-8")]
def bad(x, y, n=13):                                    # 字符级 n-gram 交集：中文按字切，见注
    gx = {tuple(x[i:i+n]) for i in range(max(0, len(x)-n+1))}
    gy = {tuple(y[i:i+n]) for i in range(max(0, len(y)-n+1))}
    return len(gx & gy) > 0
ev  = [json.loads(l)["instruction"] for l in open("pool20_eval.jsonl", encoding="utf-8")]
w6  = [r.get("instruction", r.get("prompt", "")) for r in
       (json.loads(l) for l in open("E:/study/posttrain/llm-posttraining-lab/m2-sft/data/instruct.jsonl",
encoding="utf-8"))]
kept = [p for p in pairs if not any(bad(p["prompt"], s) for s in ev + w6)]
print(f"before={len(pairs)} after={len(kept)} dropped={len(pairs)-len(kept)}")
by = {}
for p in kept: by.setdefault(p["prompt"], []).append(p)
keys = list(by); random.shuffle(keys); cut = max(1, int(round(len(keys)*0.1)))
hold = [x for k in keys[:cut] for x in by[k]]; tr = [x for k in keys[cut:] for x in by[k]]
for name, rows in [("pref_train.jsonl", tr), ("pref_holdout.jsonl", hold)]:
    with open(name, "w", encoding="utf-8") as f:
        for r in rows: f.write(json.dumps(r, ensure_ascii=False)+"\n")
print("train =", len(tr), "holdout =", len(hold),
      "| prompt overlap =", len({r['prompt'] for r in tr} & {r['prompt'] for r in hold}))
```

完成判据：`dropped` 数字打印出来（**看到 0 先别高兴，插一行 `print(len(ev)+len(w6), len(kept))` 确认函数真的在被调用**）；train + holdout = kept；prompt 交集打印为 0；held-out ≥100 对（不足就把比例升到 12% 并在 NOTES 记）。

> 注：中文按字符切 13-gram 偏严（13 个汉字很容易撞车）。若 dropped 高得离谱（>30%），把 n 调到 16-18，**并记录调后的 n 与 drop 率，别悄悄把去污染关掉**。评测集泄漏任何形式的都是判零，宁可少数据。

任务 3：手写 mini-DPO 并与 DPO 语义对拍（约 160 分钟，加分项但强推）

玩具设定：词表 `V=6`，policy 与 ref 各是一个 `V×V` 矩阵（`logits = x @ W`），`log π(y|x)` 取 softmax 后第 y 类的对数概率。**全 float64**（Week1 的对拍纪律：32 位上这点差距会被噪声吞掉）。

```
# week08/day4/mini_dpo.py — 两偏好对 + 两个玩具网络的 DPO: 语义对拍 + 梯度校验
import numpy as np
rng = np.random.default_rng(0); V, BETA, LR = 6, 0.1, 0.5     # seed 写死: 断言通过率必须可复现
W   = rng.normal(size=(V, V))*0.3                              # 可训练的 policy
REF = rng.normal(size=(V, V))*0.3                              # 冻结的 ref
PAIRS = [dict(x=np.eye(V)[0], w=2, l=0), dict(x=np.eye(V)[1], w=4, l=1)]   # one-hot 输入
def softmax(z):
    z = z - z.max(); e = np.exp(z); return e/e.sum()
def logpi(Wm, x, y):                    # 玩具里「整句 logprob」退化成单 token 的 log softmax
    return float(np.log(softmax(x @ Wm))[y])
def margins(Wm):                        # r̂ = β(logπ_θ − logπ_ref); margin = r̂(chosen) − r̂(rejected)
    return np.array([BETA*((logpi(Wm,p["x"],p["w"])-logpi(REF,p["x"],p["w"]))
                           -(logpi(Wm,p["x"],p["l"])-logpi(REF,p["x"],p["l"]))) for p in PAIRS])
def loss_grad(Wm):                      # d softmax/d logit = diag(d) − d dᵀ, 再乘 β 与 σ(−m)
    ms = margins(Wm); L = 0.0; G = np.zeros_like(Wm)
    for p, m in zip(PAIRS, ms):
        s = 1.0/(1.0+np.exp(-m)); L += -np.log(np.clip(s, 1e-12, 1.0)); d = softmax(p["x"] @ Wm)
        G += (s - 1.0) * BETA * (np.outer(p["x"], np.eye(V)[p["w"]]-d) - np.outer(p["x"], np.eye(V)
[p["l"]]-d))
    return L/len(PAIRS), G/len(PAIRS)
def dpo_ref_impl(m):                    # 独立第二实现: 只做 σ(m) 上的 BCE, 与上式共用 margin
    return float(-np.mean(np.log(np.clip(1.0/(1.0+np.exp(-np.asarray(m, np.float64))), 1e-12, 1.0))))
def finite_diff_check(Wm, eps=1e-6):    # 手写梯度 vs 中心差分
    _, G = loss_grad(Wm); worst = 0.0
    for _ in range(5):
        i, j = rng.integers(0, V, 2)
        Wp, Wq = Wm.copy(), Wm.copy(); Wp[i, j] += eps; Wq[i, j] -= eps  # 各建副本, 别复用同一个改两次
        worst = max(worst, abs((loss_grad(Wp)[0]-loss_grad(Wq)[0])/(2*eps) - G[i, j]))
    return worst

m0 = margins(W).mean()          # 取标量基线; 直接留数组的话下面那句 assert 会变成"数组的真假"而报 ambiguous
truth
W0 = W.copy()
for step in range(800):
    L, G = loss_grad(W); W = W - LR*G
    if step % 200 == 0: print(f"step {step} loss {L:.6f} margin {margins(W).mean():+.4f}")
assert margins(W).mean() > m0, "margin 没上升: 梯度符号或 lr 有问题"
for k, p in enumerate(PAIRS):                          # 判据 1: chosen 概率上升
    a, b = np.exp(logpi(REF, p["x"], p["w"])), np.exp(logpi(W, p["x"], p["w"]))
    print(f"pair{k} chosen prob {a:.4f} -> {b:.4f}"); assert b > a, "chosen 概率未升"
print("margin %.4f -> %.4f" % (m0, margins(W).mean()), "| W 改动量", round(float(np.abs(W-W0).max()), 3))
assert abs(loss_grad(W)[0] - dpo_ref_impl(margins(W))) < 1e-12, "与独立实现 loss 不一致"   # 判据 2
err = finite_diff_check(W); print("gradcheck max_err = %.3e" % err); assert err < 1e-5      # 判据 3
print("MINI-DPO OK")
```

完成判据：打印 `MINI-DPO OK`，且屏幕上出现四组数：平均 margin 前/后、两个 pair 的 chosen 概率前/后、`gradcheck max_err`（<1e-5）、loss 对拍差（<1e-12）。**800 步 margin 不涨就先把 LR 降一半再判，别加步数硬撑**——不涨通常意味着梯度符号反了。

> 注：这套玩具**对拍的是「语义」而不是数值**——同一个 margin 定义、同一个 σ 上的 BCE、同一个 β 的位置。真要与 TRL 数值对拍（Day 5 有余力再做）：换真模型的两个 ckpt，用 `DPOTrainer` 只跑 1 step，把它打印的 loss 与你独立算的 margin-BCE 对上；**版本对不上是常态，能对到小数点后 3 位就停手**，别为 1e-6 耗掉晚上。

> 注：`finite_diff_check` 里最容易犯的错是**复用同一个矩阵改两次不复原**，那算出来的不是中心差分。这类错误只有对拍才抓得到——**它是 Week1 数值梯度裁判真正派上用场的地方**，比一次写对更值。

任务 4：收工归档（约 30 分钟）

把一致率三个数字、去污染 drop 数、mini-DPO 四组数值一次写进 `annotation-report.md` 与 NOTES.md。

完成判据：三个一致率百分数、drop 条数、`MINI-DPO OK` 的打印值在文件里各自可见（grep 得到行号）；`git status --short` 无未跟踪产物。

今日验收：一致率有数 + train/holdout 交集 0 + `MINI-DPO OK` + 报告成稿。

Day 5 • 周五：DPOTrainer 正式训练

今日总览（6h）：上卡日。**β=0.1、lr=5e-6（简报硬数字，不许临场改）**，起点是第 6 周 SFT 的 merge 权重。上午把显存账与卡时账算清，下午把 1.5B（或 0.5B）的 DPO 跑完并确认三个指标动起来了。晚上关机。

任务 1：显存账 + 开卡前算账（约 40 分钟）

先在 NOTES.md 算完两条路线再动手：

路线	权重份数	1.5B 显存量级手算	结论
全量 DPO	policy + ref = 2 份，各 ≈3.0GB	权重 6GB + policy 梯度 ≈3GB + AdamW 态 8 字节/参 ≈12GB + 激活 3-5GB	**≈24GB 顶格**，必须 gradient checkpointing + paged/8bit AdamW + batch=1
LoRA DPO（今天走这条）	基座 1 份冻结 ≈3.0GB	基座 3GB + adapter 训练态（千分之几参数）+ 激活 2-4GB	**≈8-12GB 很舒适**，ref 用「关掉 adapter」得到，不额外占一份

```
cat > E:/study/posttrain/week08/day5/run.sh <<'EOF'
source /etc/network_turbo
pip install -U "trl>=0.11" "transformers>=4.45" peft accelerate wandb -i
https://pypi.tuna.tsinghua.edu.cn/simple
cd /root/autodl-tmp/week08 && python train_dpo_lora.py 2>&1 | tee dpo.log
EOF
```

完成判据：NOTES.md 有那张两行显存表（含手算数字，不是「大概十几 G」）；`run.sh` 已写好；写下今日卡时目标 5h ≈¥8-12。

> 注：为什么不用全量？**不是跑不起，是跑不起「一天调一轮」**。全量 1.5B 要在 24G 里同时供四份东西（两套权重、一套梯度、一套优化器态），显存顶格意味着 batch 压到 1、序列压到 512，一次实验 3 小时起。LoRA 版把可训参数降到千分之几、ref 复用基座，一次 1 小时——**本周你要的是看清曲线，不是刷最高分**。面试问「为什么不用全量」，答案就是这张表。

任务 2：LoRA DPO 训练（约 180 分钟）

`week08/day5/train_dpo_lora.py`（**本地写好再传实例**，别在网页终端里现敲）：

```
import json, torch
from transformers import AutoModelForCausalLM, AutoTokenizer
from peft import LoraConfig
from trl import DPOTrainer, DPOConfig

MODEL = "/root/autodl-tmp/sft-15b-merged"
tok = AutoTokenizer.from_pretrained(MODEL)
load = lambda p: [json.loads(l) for l in open(p, encoding="utf-8")]
def fits(e):                                        # 预处理阶段按 token 长度过滤，不做截断(见注)
    a = len(tok(e["prompt"]+e["chosen"], truncation=False)["input_ids"])
    b = len(tok(e["prompt"]+e["rejected"], truncation=False)["input_ids"])
    return max(a, b) <= 1024
tr = [e for e in load("pref_train.jsonl") if fits(e)]
ev = [e for e in load("pref_holdout.jsonl") if fits(e)]
print("after filter: train =", len(tr), " test =", len(ev))

model = AutoModelForCausalLM.from_pretrained(MODEL, torch_dtype=torch.bfloat16)
model.enable_input_require_grads()                  # LoRA + gradient checkpointing 需要它
cfg = LoraConfig(r=16, lora_alpha=32, lora_dropout=0.05,
                 target_modules=["q_proj","k_proj","v_proj","o_proj","gate_proj","up_proj","down_proj"])
args = DPOConfig(output_dir="/root/autodl-tmp/out/dpo-15b-lora",
                 beta=0.1, learning_rate=5e-6,                    # 简报硬数字
                 per_device_train_batch_size=4, gradient_accumulation_steps=4,   # 等效 batch 16
                 num_train_epochs=1.0, bf16=True, gradient_checkpointing=True,
                 optim="paged_adamw_8bit", logging_steps=10, save_steps=150,
                 seed=42, data_seed=42, report_to="wandb")
trainer = DPOTrainer(model=model, ref_model=None, args=args, peft_config=cfg,   # PEFT 模式: ref = 关掉
adapter
                     train_dataset=tr, eval_dataset=ev, processing_class=tok)
trainer.train(); trainer.save_model("/root/autodl-tmp/out/dpo-15b-lora/final")
```

`bash run.sh` 起跑（**中途 Ctrl+C 之后也别忘了关机**）。

> 注：**这里刻意过滤而不是截断**。DPO 的截断语义很容易踩雷：默认截法可能从**末尾**切，也就是把 assistant 的答案切掉、只留 prompt 头部——模型于是学「这个 prompt 后面什么都没有」，loss 看着在降、margin 不动，你还以为是 lr 问题。口诀类样本本就短（≤200 token），被过滤的是极少数。**真要截，保住答案，宁可截 prompt 的头。**

完成判据：跑满 ≥80% 计划步数（1 epoch、等效 batch 16、约 1100 对 → **预期量级 60-80 step**，以实际打印为准）；wandb 或 `dpo.log` 里 loss 从初值往下走、**含 `margin` 的那个指标上行**；`nvidia-smi` 峰值 ≤20GB（截图 `day5/gpu.png`）；final adapter 目录非空。

> 注：loss 不降的排查顺序（简报 §6 的通用顺序落到 DPO）：① **margin 是否恒 ≈0**（是 → 数据没信号，回查 Day 4 阈值与人判一致率，不是超参问题）② `trainable params` 打印出来**必须 <5% 总量**，否则 LoRA 没生效、你在训全量 ③ 数据列名是否被 TRL 静默忽略（打印一条 batch 的实际拼接文本）④ 才轮到 lr。限时 40 分钟，不通就带着日志进明早，别熬夜。

任务 3：训练健康检查（约 80 分钟）

1. 打开 wandb（离线就 `export WANDB_MODE=offline` 跑，之后 `wandb sync <目录>` 补传），截图三条：`loss`、含 `margin` 的指标、含 `accuracies` 的指标。
2. 三个问题各 ≥2 行写进 NOTES.md，**必须带具体数字**：① 训练集 `rewards/accuracies` 终值多少？与 Day 6 held-out 差多少（训练集明显更高 = 记忆而非偏好）？② chosen 与 rejected 的 logprob 趋势各自如何，是否出现「rejected 也在涨、只是慢一点」？③ loss 有没有短暂上翘后又下降?

完成判据：三张截图存在；三问各有带数字的回答；`accuracies` 有一个可读的终值。

任务 4：产物备份与关机（约 60 分钟）

```
cd /root/autodl-tmp/out/dpo-15b-lora/final && zip -r /root/autodl-tmp/dpo-lora.zip .
cd /root/autodl-tmp/llm-posttraining-lab && git add m2-sft/dpo && git commit -m "week8 dpo run" && git push
```

完成判据：GitHub 能看到 `m2-sft/dpo/`（adapter 权重本体太大不进 git，进 zip，README 写明它在哪）；控制台「已关机」；NOTES 记今日卡时（≤5h）与实际花费；NOTES.md 补三行（今天最卡的点 / margin 与 accuracy 终值 / 明天评测看什么）。

> 注：`/root/autodl-tmp` 是持久盘，`/root` 下内容可能随镜像重置丢。今天所有 ckpt、日志、zip 都要在 `autodl-tmp` 下 **并且 git 里有一份元信息**。别把「跑完了」当成「东西还在」。

今日验收：训练跑完 + 三条曲线截图 + 三问有答案 + adapter 双备份 + 已关机。

Day 6 • 周六：两份评测 + reward hacking 观察

今日总览（7h）：本周收口。**DPO 最难的部分不是训，是证明它变好了**——held-out 偏好对准确率只说明「模型认同标注器的判断」，20 题盲评才说明「人看着是不是更好」。上午算 held-out 准确率；中午定死盲评规则并跑 20 题；下午专门观察 reward hacking（长度分布 + 套话计数）。

任务 1：held-out 偏好对准确率（约 90 分钟）

只在 assistant span 上算长度归一化 logprob，`chosen > rejected` 的比例即准确率：

```
# week08/day6/eval_pref_acc.py
import json, torch
from transformers import AutoModelForCausalLM, AutoTokenizer
MODEL = "/root/autodl-tmp/out/dpo-15b-lora/merged"        # LoRA 先 merge_and_unload 再存到这里
tok = AutoTokenizer.from_pretrained(MODEL)
model = AutoModelForCausalLM.from_pretrained(MODEL, torch_dtype=torch.bfloat16).to("cuda").eval()

def encode(prompt, answer):
    full = tok.apply_chat_template([{"role":"user","content":prompt},
                                    {"role":"assistant","content":answer}], tokenize=True,
add_generation_prompt=False)
    head = tok.apply_chat_template([{"role":"user","content":prompt}], tokenize=True,
add_generation_prompt=True)
    assert full[:len(head)] == head, f"前缀不一致 {len(head)}: BPE 把边界合并了"
    return full, [0]*len(head) + [1]*(len(full)-len(head))   # 只 assistant span 计入

@torch.no_grad()
def norm_logprob(ids, mask):
    ids = torch.tensor([ids], device=model.device)
    m = torch.tensor([mask[1:]], device=model.device, dtype=torch.float32)          # 与 t+1 对齐
    lp = torch.log_softmax(model(input_ids=ids).logits[0, :-1, :].float(), dim=-1)   # bf16 模型 → fp32 求
和
    tl = lp.gather(-1, ids[0, 1:].unsqueeze(-1)).squeeze(-1)
    return float((tl*m).sum()) / max(1.0, float(m.sum()))    # 长度归一化，避免长答案天然占优

pairs = [json.loads(l) for l in open("pref_holdout.jsonl", encoding="utf-8")]
hit = sum(norm_logprob(*encode(p["prompt"], p["chosen"])) >
          norm_logprob(*encode(p["prompt"], p["rejected"])) for p in pairs)
print(f"held-out pref acc = {hit}/{len(pairs)} = {hit/len(pairs):.2%}")
```

完成判据：打印 `held-out pref acc = X/Y = Z%`，Y 与 Day 4 的 held-out 对数一致；**Z ≥60%** 记「学到了偏好信号」（预期量级 60-75%）；**Z <55%** 记训练无效，回 Day 5 任务 2 的排查顺序；**Z ≥95% 反而要怀疑 held-out 泄漏**，回查 Day 4 切分。

> 注：那句 `assert full[:len(head)] == head` 是全周最实用的一行。BPE 会把「prompt 结尾的一个字符」与「assistant 起始标记」**合并成一个 token**，于是 head 不再是 full 的前缀，mask 整体错位一格——loss 正常、准确率却怪。**报错是设计给你的**：真撞上就把边界那格 mask 置 0，并记下发生频次。这是面试问「chat template 踩过什么坑」最硬的回答格式。

任务 2：20 题人工盲评（约 140 分钟）

1. **先写死规则**（`m2-sft/dpo/blind-rules.md`），规则写完才开始评：
   - 题面用 Day 1 封存的 `pool20_eval.jsonl` 20 题；SFT-only 与 SFT+DPO 各生成 1 条（temperature=0.7、seed=2026、max_tokens=200）。
   - A/B 左右顺序按 `random.seed(2026)` 随机，评分者看不到哪栏来自哪个模型。
   - 三轴各 0-2 分：**准确性**（词义/拼写/读音对不对）、**助记性**（能否一眼想起这个词）、**简洁性**（≤25 字 2 分，26-40 字 1 分，>40 字 0 分），总分 0-6。
   - 两栏总分差 ≤1 记平局；全部评完揭盲，统计胜/平/负。
   - **揭盲后随机抽 4 题重评一次；两次判定不一致 ≥2 题，本轮盲评作废重评。**

2. 生成两侧输出（同一实例上 merge LoRA 后采样，**复用 Day 5 那台，别为这步单独开卡**），导出 `blind20.md`（评分列先留空，机器名不进文件）。
3. 评完揭盲，写 `blind20-result.md`：胜/平/负、三轴各自均分、一胜一负两个典型例子。

完成判据：`blind-rules.md` 创建时间早于任何评分记录（**这条要能被 git log 证明**：规则先单独 commit 一次）；20 题全部有分（空白 = 没评）；揭盲一致性检查通过（不一致 ≤1/4）；结果表里 DPO 版 **胜 ≥8 且 负 ≤4** 记「盲评通过」；未达标就照实写「盲评未通过」，**不许改规则重跑一次当作通过**。

> 注：盲评未通过是**很有价值的结论而不是失败**。它可能说明偏好数据里「机器分高」≈「更长更规整」，而人判的是「更好记」。这时 Day 3 任务 1 的 rubric 权重就是你的诊断对象，写进 README 的失败案例一节。周 14 的「失败案例复盘」清单最缺的就是这种带数字的真案例。

任务 3：reward hacking 观察（约 100 分钟）

对同一 200 条 prompt（从 600 池取、去掉 held-out）分别用 SFT 与 SFT+DPO 各生成一次，落 `gen_sft.jsonl` / `gen_dpo.jsonl`（每行 `{"text": ...}`）：

```
# week08/day6/length_and_filler.py
import json, numpy as np, matplotlib
matplotlib.use("Agg"); import matplotlib.pyplot as plt
def load(p): return [json.loads(l)["text"] for l in open(p, encoding="utf-8")]
sft, dpo = load("gen_sft.jsonl"), load("gen_dpo.jsonl")
for tag, ts in (("SFT", sft), ("DPO", dpo)):
    a = np.array([len(t) for t in ts], dtype=np.float64)
    print(f"{tag}: n={len(a)} mean={a.mean():.1f} p50={np.percentile(a,50):.0f} "
          f"p90={np.percentile(a,90):.0f} max={a.max():.0f}")          # p50/p90 比均值抗离群
    plt.hist(a, bins=30); plt.title(tag); plt.savefig(f"len_{tag}.png"); plt.close()
CLICHES = ["我完全理解你的感受", "希望这能帮到你", "作为一个人工智能", "让我们一起", "废话不多说"]
for tag, ts in (("SFT", sft), ("DPO", dpo)):
    n = max(1, len(ts))
    print(tag, {c: round(sum(c in t for t in ts)/n*100, 2) for c in CLICHES if any(c in t for t in ts)})
```

在 `m2-sft/dpo/hacking.md` 回答三问，各 ≥4 行、**至少一条带可验证数字**：① DPO 有没有变长、长了多少？② 变长是好事还是骗分？③ 如果我故意把「更长=更好」写进 rubric，我会看到什么？

完成判据：三问齐全且带数字；两张直方图 png 存在；两侧套话百分比都打印出来。**若 DPO 平均长度比 SFT 涨 >25% 而盲评总分没涨**，这就是本周找到的第一个真实 reward hacking，**必须在报告里单列一节，不许埋在附注**。

> 注：DPO 的 reward hacking 与 RLHF 形态不同：DPO 的奖励是隐式的，**被攻击的不是 RM 而是标注器**。典型链条：rubric 里 `concise` 只占 0.15，而 `accuracy` / `mnemonic` 各 0.35 且打分器倾向给「解释得更完整」的答案高分 → 模型学到「写长一点像更准确」。所以今天必须同时看长度与套话两条，单看长度会误判。**这条链条就是面试「reward hacking 出现过吗」的标准答案格式：现象 → 数字 → 根因（rubric 权重）→ 下一步怎么办。**

任务 4：README、周复盘与归档（约 90 分钟）

补齐 `m2-sft/dpo/README.md`，**按简报 §4 第 10 条写清诚信边界**：

```
- 复现自官方：TRL DPOTrainer 的训练配置(beta=0.1, lr=5e-6)，改编自 trl 示例
- 我自己写：score.py / pair.py / split_clean.py / mini_dpo.py / eval_pref_acc.py / 盲评生成器
- 数据来源：第 7 周自建 english-tutor-instruct 的 600 条 instruction；打分器 <模型名>，提示词 v1(10/29)
- 质检：100 条人工复核一致率 = __% ；held-out 准确率 = __% ；盲评 胜/平/负 = __/__/__
- 未做：未训 RM，未跑 PPO/GRPO，β 只跑了 0.1 一档，未做多种 seed
```

提交 DPO 推导手稿拍照、两份评测表、曲线截图，`git push`。再写 `week08/day6/review.md` 五块：硬产出清单（路径）/ 本周最耗时的一件事 / 盲评结论一句话 / 一致率数字 / 最想补的一个洞。

完成判据：README 五段齐全（**「我自己写」这段不许省略**）；repo 里能翻到 `dpo-derivation.md`、`review100.md`、`blind20-result.md`、`hacking.md` 四个文件；`git status` 干净；中文 jsonl 以 UTF-8 存且渲染无乱码；review.md 五块齐全。

今日验收：held-out 准确率有数 + 盲评规则与结果表 + hacking.md 三问 + README 五段 + review.md。

Day 7 • 周日：补漏 + 闭卷复测 + 总验收 + 下周预习

今日总览（4.5h）：不排新实验。上午清两个洞，中午闭卷复测，做 7 项总验收，下午预习下周 PPO/GRPO。**今天最忌把「再训一轮 DPO 看能不能更高」排进来**——那是下周之后的事。

任务 1：补漏（约 90 分钟）

打开 `week08/gaps.md`。**规则：最多补 2 个洞，贪多必失。** 优先这两类：

1. DPO 推导卡住的那一步（通常在第 2 步「变分 + 拉格朗日」或第 4 步「Z 相消」）。补法：重推一遍写进 `day7/re推导.md`，写完闭卷讲 3 分钟并录音自查。
2. held-out 与训练集 accuracy 差距解释不了。补法：手工拆开 5 条 held-out pair 逐条判，写 5 行进报告，结论落在「记忆 vs 偏好」二选一。

其余洞**原样映射到后续周**，今天不硬补。

完成判据：两份产出至少一份成稿（≥5 行实质内容）；被补的两条在 gaps 标 `[x]` 并写「怎么补的」。

任务 2：闭卷复测（约 50 分钟，别跳）

白纸三题，写完与 Day 2 推导、Day 3/4 的报告对照：

1. 写 BT loss 与 DPO loss 全式，指出 implicit reward 定义，并说明推导里配分函数 Z(x) 怎么消失（**三句话以内**）。
2. 一句话各讲清 IPO / KTO / SimPO 解决 DPO 的什么问题。
3. 「你的偏好数据有多少对？held-out 怎么切的？人判一致率多少？低于某个值你怎么办？」——按本周真实数字答。

完成判据：第 1、2 题全对（Z 相消答成「因为 Z(x) 只与 prompt 有关、在同 prompt 两条答案之差里出现两次、符号相反」也算对）；**第 3 题四个数字全部报得出**，报不出就今天回 Day 3/4 补齐——**这题面试必问，答不上比推导错更致命。**

> 注：这是全周唯一真正检验「补上了没有」的环节，`gaps.md` 上打 `[x]` 不算数。复测结果（对/错/半对）写进 `review.md`。

任务 3：总验收（约 30 分钟）

#	验收项	判据
1	Bradley-Terry	`BT CROSSCHECK OK` 打印；能写「r 常数平移不影响 BT」的理由
2	DPO 推导	六步齐全；闭卷 3 分钟讲完不打卡
3	偏好数据	`pref_pairs_raw.jsonl` 在 1000-2000 对；标注提示词 v1 已存档；采样与打分 seed 写死
4	人工质检	100 条人判全部填完；一致率、强一致率、差不多占比三个数字进报告
5	DPO 训练	β=0.1、lr=5e-6 落实；loss 下行 + margin 上行的曲线截图；adapter 双备份
6	评测	held-out 准确率有数（≥60% 合格，<55% 记未通过并写原因）；盲评规则早于评分（git log 可证）且已揭盲
7	hacking 观察	`hacking.md` 三问带数字；长度直方图；README 含「我自己写 / 复现自官方」声明

≥6 项通过 = 本周合格；不足 6 项，把缺的写进下周 Day 1 的前 2 小时（**下周 Day 1 的理论内容不许为它让路，M3 的交付时间是硬约束**）。

完成判据：7 行逐行填「通过 / 未通过 + 证据所在路径」，全表无空白格；未通过项已在 `gaps.md` 建了带具体日期的条目。

任务 4：下周预习（约 30 分钟）

三件事各写 5 行进 `notes/preview-week09.md`：

1. 把 Day 2 那张「SFT / DPO / PPO / GRPO 对照」表补全你没把握的格子——**下周 Day 1 就从这些格子开始**。
2. 读 DeepSeekMath 里 GRPO 目标式那一段，抄下 `Â_i = (r_i - mean)/std` 并写一句「分母那个 std 为什么必要」。
3. 看 `github.com/volcengine/verl` 的 README 安装节，**记下它要求的 vLLM / torch / CUDA 版本组合**，写进 gaps。

> 注：第 3 件看着像预习，实际是**给下周 Day 4 排风险**。verl 装不动时，你有一整周准备降级路线（TRL `GRPOTrainer` + 0.5B + 小样本），而不是下周四晚上对着红色报错想办法。

完成判据：三条各 ≥5 行且落盘在 `notes/preview-week09.md`；Day 2 对照表未填格数 = 0；verl 那行写明具体版本号（写不出就标「README 未给版本矩阵」并附当天访问日期，别留空）。

任务 5：归档与账面收口（约 70 分钟）

1. DPO 推导**纸上手稿**拍照 → `m2-sft/dpo/derivation-photo.jpg`，`git push`。
2. `week08/gaps.md` 按「周 9 / 周 11 / 周 12 / 周 14」四栏重排，每条写成「洞 → 哪天 → 什么动作」，**不许留没有归期的条目**。
3. 记四对账：卡时上限 12h / 实际 **h；GPU ¥25 / 实际**；API ¥20 / 实际 __；计划 42h / 实际 __h。写进 `day7/review.md`。

完成判据：手稿照片在 repo 可见；gaps 每条有归期（**带「以后再说」字样的算未完成**）；四对账面数字齐全。

> 注：账面收口每次只做 5 分钟你就不会做。它是简报里「开卡前写好启动脚本、跑完即关」的另一半——**没有事后对账，你永远不知道超支来自哪一次重跑**。周 9 预算更紧（¥35 且要短租 A100），本周不养成对账习惯，下周就会凭感觉花钱。

今日验收：两个洞有产出 + 复测三题（第 3 题四数字齐全）+ 验收表 ≥6 项打勾 + 预习三条 + gaps 全部有归期、账面四对落纸。

本周暴露的洞 → 后续哪一周补

洞	归到	那天的动作
margin 不涨 / held-out 上不去，怀疑偏好标注本身弱	**周 9 · Day 3 规则奖励**	GRPO 用可验证奖励（对/错）换掉「LLM 打分」，同一件事换裁判再测
长度偏置只观察到、没修	**周 9 · Day 6 任务 2**	「只给长度奖励」的小实验，正面制造 hacking 再诊断
没训过 RM，RM 只停在结构图	周 12 · 自建教学评测集	时间允许就用本周 pair 训一个标量 RM 做 rubric 对照（不排进周 9-11）
β 只跑 0.1，没消融	周 11 · tutor DPO	复用本周管线时顺带跑 β=0.01 / 0.1 / 0.5 三档
IPO / SimPO 只到能讲、没跑	周 11 · 对齐阶段可选	有富余卡时再把 KTO 用于单边标签（pair 拆成好/坏两类即可）

三、常见问题与纪律

- **某天没完成怎么办？** 不熬夜追。按「天-任务号」写进 `week08/gaps.md`，Day 7 就是给它们准备的。连续两天崩则砍 Day 4 任务 3（mini-DPO）与 Day 2 任务 5，**保 Day 3 标注、Day 4 复核、Day 5 训练、Day 6 评测**。
- **卡住多久该求助？** 单点 40 分钟：写 NOTES → 搜报错原文与 trl 仓库 issue → 再 20 分钟无解就带着日志跳过并在 gaps 立条目。DPO 的坑大多是**版本 API 不匹配**，搜的时候一定带上 `trl.__version__`。
- **钱**：卡时 ≤12h、GPU ≤¥25、API ≤¥20，合计上限 ¥50。超支两个常见来源：忘关机（看 AutoDL 计费明细，一天内能定位）、批量打分写成无限重试。**Day 3 任务 1 那张算账表就是防这两件事的。**
- **数据合规**：本周偏好数据源自第 7 周你自己发布的公开数据集；**打分器的输出一律视为「模型意见」而非事实**，人工复核过的 100 条要标「已人审」；明日记忆的真实日志若混入，必须已在第 7 周完成匿名化，**本周不许重新引入任何用户可识别信息**。

- **诚信边界**：README 里「复现自官方 / 我自己写」两段是硬要求（简报 §4 第 10 条）。TRL 示例改动、mini-DPO 与评测脚本自己写、标注提示词自己设计——分清楚，这是实习面试最容易被追问的点。
- **别提前用 AI 生成代码**：允许 AI 帮你解释报错、讲清变分那一步、review 你写完的文件；**不允许**它整段生成 `train_dpo_lora.py` 或推导手稿。推导的价值就在你卡住的那 20 分钟里。
- **今天不碰 GRPO / PPO / verl**。想提前练的手感写进 gaps，下周 Day 1 再放出来。